Tribhuvan University

Faculty of Humanities & Social Sciences

OFFICE OF THE DEAN

System Analysis & Design

BCA — Semester 3 — 2020 — Answer Sheet

Subject Code: CACS203

Group B

Attempt any SIX questions. [6×5=30]

2. What is system? What are the phases of SDLC? Explain briefly.

[1+4]

Answer: System: A system is a collection of interrelated components that work together to achieve a common goal. It accepts inputs, processes them, and produces outputs. A system has boundaries, interacts with its environment, and is composed of subsystems that are themselves systems. Examples include information systems, operating systems, and ecosystems. Characteristics of a system include components, interrelatedness, boundaries, purpose, and environment.

Phases of SDLC (System Development Life Cycle):

1. **Planning:** Identifying the need for a new system, conducting feasibility studies (technical, economic, operational), defining project scope, and obtaining approval. The project plan, schedule, and budget are created.
2. **Analysis:** Gathering detailed requirements from stakeholders through interviews, surveys, and observation. Current system is studied to identify problems and opportunities. Requirements are documented in SRS (Software Requirements Specification).
3. **Design:** Creating system architecture, database design, interface design, and detailed module specifications. Design documents include DFDs, ERDs, data dictionaries, and prototype screens.
4. **Implementation (Development):** Actual coding and programming based on design specifications. Developers write code, perform unit testing, and integrate modules. This is the phase where the system is actually built.
5. **Testing:** Verifying that the system meets requirements and is free of defects. Includes unit testing, integration testing, system testing, and user acceptance testing (UAT).
6. **Deployment:** Installing the system in the production environment, training users, and migrating data from the old system. Can be done through direct cutover, parallel running, phased implementation, or pilot approach.
7. **Maintenance:** Ongoing support, bug fixes, enhancements, and updates after deployment. Types include corrective, adaptive, perfective, and preventive maintenance.

3. What do you mean by planning? Write the process of planning for Information System Development Project.

[1+4]

Answer: Planning: Planning is the process of defining goals, establishing strategies, and developing plans to coordinate activities. In the context of information systems, planning involves determining what needs to be done, how it will be done, who will do it, when it will be done, and what resources are required. Effective planning reduces uncertainty, minimizes waste, and establishes standards for controlling project progress.

Process of Planning for Information System Development Project:

1. **Define Project Scope and Objectives:** Clearly state what the project will accomplish, identify stakeholders, and establish measurable objectives. Define what is in scope and what is out of scope.
2. **Conduct Feasibility Study:** Evaluate the project from multiple perspectives:
• **Technical feasibility:** Can the technology support the proposed solution?
• **Economic feasibility:** Is the project financially viable? (cost-benefit analysis, ROI)
• **Operational feasibility:** Will the system be used effectively within the organization?
• **Schedule feasibility:** Can the project be completed within the required timeframe?
3. **Estimate Resources:** Determine human resources (analysts, developers, testers), hardware, software, and budget requirements needed for the project.
4. **Develop Work Breakdown Structure (WBS):** Break down the project into smaller, manageable tasks and subtasks. Each task should have clear deliverables and responsible persons.
5. **Create Project Schedule:** Estimate time for each task, identify dependencies, and create a timeline using Gantt charts or PERT/CPM networks. Identify the critical path.
6. **Identify Risks:** Identify potential risks (technical, organizational, external), assess their probability and impact, and develop mitigation strategies.
7. **Prepare Project Plan Document:** Compile all planning information into a formal project plan including scope, schedule, budget, resources, risk management plan, and communication plan.
8. **Obtain Approval:** Present the project plan to management and stakeholders for review and approval before proceeding.

4. Define process modeling. Explain DFD with example.

[1+4]

Answer: Process Modeling: Process modeling is the graphical representation of the processes (functions or activities) that capture, manipulate, store, and distribute data between a system and its environment and among system components. It helps analysts understand how data flows through an organization and how it is transformed. Data Flow Diagram (DFD) is the most common tool used for process modeling.

Data Flow Diagram (DFD):
A DFD is a graphical representation of the flow of data through an information system. It shows how data is input, processed, stored, and output by the system. DFDs can be drawn at different levels of detail (context diagram, level 0, level 1, etc.).

Components of DFD:
1. **Process (Circle/Bubble):** Represents an activity or function that transforms data. Named with a verb phrase (e.g., "Validate Order", "Calculate Payroll").
2. **External Entity (Rectangle):** Represents sources or destinations of data outside the system being modeled (e.g., "Customer", "Bank", "Supplier").
3. **Data Store (Open Rectangle/Parallel Lines):** Represents a repository where data is stored (e.g., "Customer Database", "Order File", "Inventory Master").
4. **Data Flow (Arrow):** Represents the movement of data between processes, entities, and data stores. Labeled with the data name.

Example — Online Order Processing System:
Context Diagram (Level 0):
External Entities: Customer, Warehouse, Bank
Process: Order Processing System
Data Flows: Order Details (Customer → System), Payment Info (Customer → System), Order Confirmation (System → Customer), Shipping Request (System → Warehouse), Payment Authorization (System → Bank)

Level 1 DFD:
Processes:
• P1: Receive Order (Customer → P1 → Order File)
• P2: Validate Payment (P1 → P2 → Bank → P2 → Payment File)
• P3: Process Order (P2 → P3 → Order File, Inventory File)
• P4: Generate Invoice (P3 → P4 → Customer)
• P5: Ship Order (P3 → P5 → Warehouse → Customer)

5. List and explain the skills and responsibilities of project manager.

[2.5+2.5]

Answer: Skills of a Project Manager:

1. **Leadership:** Ability to inspire, motivate, and guide team members toward achieving project goals. Includes delegation, conflict resolution, and decision-making.
2. **Communication:** Strong verbal and written communication skills to interact with stakeholders, team members, and clients. Includes active listening and presentation skills.
3. **Technical Knowledge:** Understanding of the technical aspects of the project, tools, and methodologies being used. For IS projects, knowledge of databases, programming, and system design.
4. **Time Management:** Ability to prioritize tasks, meet deadlines, and manage schedules effectively using tools like Gantt charts and project management software.
5. **Risk Management:** Identifying potential risks, assessing their impact, and developing mitigation strategies to minimize project disruptions.
6. **Budget Management:** Planning, estimating, and controlling project costs to ensure the project is completed within the approved budget.
7. **Problem-Solving:** Analytical thinking and creativity to resolve issues that arise during the project lifecycle.
8. **Negotiation:** Ability to negotiate with stakeholders, vendors, and team members for resources, timelines, and scope adjustments.

Responsibilities of a Project Manager:

1. **Project Planning:** Defining project scope, objectives, deliverables, and creating detailed project plans including schedules and budgets.
2. **Resource Allocation:** Assigning the right people to the right tasks and ensuring necessary resources (hardware, software, budget) are available.
3. **Team Management:** Building and leading the project team, defining roles and responsibilities, and fostering a collaborative work environment.
4. **Stakeholder Communication:** Keeping all stakeholders informed about project progress, issues, and changes through regular meetings and status reports.
5. **Quality Assurance:** Ensuring that project deliverables meet the required quality standards and satisfy stakeholder requirements.
6. **Risk Monitoring:** Continuously tracking identified risks, identifying new risks, and implementing contingency plans when necessary.
7. **Change Management:** Managing scope changes, assessing their impact on schedule and budget, and obtaining proper approvals.
8. **Project Closure:** Ensuring all project deliverables are completed, conducting post-project reviews, documenting lessons learned, and formally closing the project.

6. Explain the guidelines to design an interface and dialogue box for e-commerce system.

[5]

Answer: Guidelines for Designing Interface and Dialogue Box for E-Commerce System:

1. **Simplicity and Clarity:** Keep the interface clean and uncluttered. Use clear labels, intuitive icons, and familiar navigation patterns. Avoid unnecessary elements that distract users from their shopping goals.

2. **Consistent Layout:** Maintain consistency in color schemes, fonts, button styles, and navigation across all pages. The header, footer, and navigation should appear the same on every page.

3. **Responsive Design:** Ensure the interface adapts seamlessly to different screen sizes (desktop, tablet, mobile). Use flexible grids, scalable images, and touch-friendly buttons.

4. **Prominent Call-to-Action (CTA):** Buttons like "Add to Cart", "Buy Now", and "Checkout" should be visually prominent using contrasting colors and appropriate sizing.

5. **Search and Filter Functionality:** Provide a visible search bar with auto-suggestions. Include filters for category, price range, brand, ratings, etc.

6. **Clear Product Information:** Display high-quality product images, detailed descriptions, prices, availability status, reviews, and shipping information clearly.

7. **Dialogue Box Design Guidelines:**
• **Confirmation Dialogues:** Use modal dialogs for critical actions (delete cart, cancel order) with clear "Confirm" and "Cancel" options.
• **Error Messages:** Display user-friendly error messages explaining what went wrong and how to fix it (e.g., "Invalid card number. Please check and re-enter.").
• **Progress Indicators:** Show loading spinners or progress bars during checkout, payment processing, or order placement.
• **Success Messages:** Provide clear confirmation after successful actions (e.g., "Order placed successfully! Order #12345").
• **Input Validation:** Validate form inputs in real-time and display inline error messages near the relevant fields.

8. **Security Indicators:** Display security badges, SSL certificates, and trusted payment icons to build user trust during checkout.

9. **Accessibility:** Follow WCAG guidelines — ensure sufficient color contrast, keyboard navigation support, alt text for images, and screen reader compatibility.

10. **User Feedback:** Provide immediate visual or auditory feedback for user actions (hover effects, button clicks, cart updates) to enhance interactivity.

7. Differentiate between system and user documentation with their applications.

[5]

Answer: Difference between System Documentation and User Documentation:

AspectSystem DocumentationUser Documentation Target AudienceDevelopers, programmers, system analysts, technical staffEnd users, operators, non-technical staff ContentSystem design, architecture, code comments, data dictionaries, DFDs, ERDs, API referencesUser manuals, help guides, FAQs, tutorials, quick start guides Technical DepthHighly technical and detailedNon-technical, simple language PurposeTo understand, maintain, and modify the systemTo learn how to use the system effectively Created BySystem analysts and developers during developmentTechnical writers and trainers after development Update FrequencyUpdated when system is modifiedUpdated when user interface or workflows change

Applications of System Documentation:

1. **System Maintenance:** Helps developers understand existing code when fixing bugs or adding features.
2. **System Enhancement:** Provides the foundation for making improvements or adding new modules.
3. **Knowledge Transfer:** Enables new team members to understand the system architecture quickly.
4. **Quality Assurance:** Serves as a reference for testers to verify that the system meets design specifications.
5. **Disaster Recovery:** Helps rebuild or restore the system in case of failures.

Applications of User Documentation:

1. **User Training:** Helps new users learn how to operate the system through tutorials and guides.
2. **Reference:** Serves as a lookup resource when users encounter unfamiliar situations.
3. **Troubleshooting:** Helps users resolve common problems without contacting support.
4. **Reducing Support Costs:** Well-documented systems reduce the number of support calls and emails.
5. **Compliance:** Some industries require documented procedures for audit and regulatory purposes.

8. Define software testing. Explain software quality assurance activities.

[1+4]

Answer: Software Testing: Software testing is the process of evaluating and verifying that a software product or application does what it is supposed to do. It involves executing a program with the intent of finding errors, defects, or gaps between actual and expected requirements. Testing can be done manually or through automated tools and includes various types such as unit testing, integration testing, system testing, and acceptance testing. The goal is to ensure the software is reliable, secure, and performs well under expected conditions.

Software Quality Assurance (SQA) Activities:

1. **Quality Planning:** Defining quality goals, standards, and criteria for the project. Creating a quality management plan that outlines how quality will be measured and ensured throughout the project.
2. **Process Definition and Improvement:** Establishing standardized development processes, methodologies, and best practices. Using frameworks like ISO 9000, CMMI, or Six Sigma to improve process maturity.
3. **Reviews and Audits:** Conducting formal reviews of requirements, design documents, and code. Audits verify that processes are being followed and deliverables meet standards.
4. **Testing:** Executing various levels of testing including unit testing (individual modules), integration testing (combined modules), system testing (complete system), and acceptance testing (user validation).
5. **Defect Tracking and Management:** Recording, categorizing, prioritizing, and tracking defects from discovery through resolution. Using tools like JIRA, Bugzilla, or Trello.
6. **Configuration Management:** Controlling changes to software artifacts through version control, change management, and release management. Ensures traceability and consistency.
7. **Metrics and Measurement:** Collecting and analyzing data on defect density, test coverage, code complexity, and customer satisfaction. Using metrics to identify trends and drive improvements.
8. **Training:** Ensuring team members have the necessary skills and knowledge to follow quality processes and use tools effectively.
9. **Documentation Control:** Maintaining accurate and up-to-date documentation throughout the software lifecycle.
10. **Customer Feedback Analysis:** Gathering and analyzing user feedback to identify quality issues and improvement opportunities.

Group C

Attempt any TWO questions. [2×10=20]

9. What are the major differences between Agile methodologies and waterfall model? Why should you use agile methodologies? Explain.

[5+5]

Answer: Major Differences between Agile and Waterfall Model:

AspectWaterfall ModelAgile Methodologies ApproachSequential and linearIterative and incremental FlexibilityRigid, difficult to change requirementsFlexible, welcomes changing requirements Customer InvolvementLimited, mainly at beginning and endContinuous throughout development DeliverySingle final product at the endWorking software delivered frequently (weeks) TestingDone after implementation phaseDone concurrently with development DocumentationHeavy documentation at each phaseLightweight, working software over documentation Risk ManagementRisks identified late, hard to mitigateRisks identified early and addressed in each sprint Team StructureFormal, role-based hierarchySelf-organizing, cross-functional teams Best ForProjects with clear, stable requirementsProjects with evolving or unclear requirements

Why Use Agile Methodologies:

1. **Adaptability to Change:** In today's fast-paced business environment, requirements change frequently. Agile embraces change and allows teams to pivot quickly based on market feedback or stakeholder input.

2. **Faster Time to Market:** Agile delivers working software in short iterations (typically 2-4 weeks). This means valuable features reach users sooner, providing competitive advantage and earlier ROI.

3. **Improved Customer Satisfaction:** Continuous customer involvement ensures the final product aligns with actual needs rather than initial assumptions. Customers see progress regularly and can provide feedback.

4. **Higher Quality:** Continuous testing, integration, and code reviews in each sprint catch defects early when they are cheaper to fix. The definition of done ensures quality standards are met.

5. **Better Risk Management:** Short iterations mean risks are identified and mitigated early. Technical and business risks surface quickly rather than at the end of a long development cycle.

6. **Transparency and Visibility:** Daily stand-ups, sprint reviews, and burndown charts provide clear visibility into project progress. Stakeholders always know the project status.

7. **Team Empowerment:** Self-organizing teams are more motivated and productive. Team members have autonomy to decide how to best accomplish their work.

8. **Reduced Waste:** Agile focuses on delivering the most valuable features first. If the project needs to be scaled back, the most important functionality is already built.

9. **Continuous Improvement:** Sprint retrospectives create a culture of learning and improvement. Teams regularly reflect on what went well and what can be improved.

10. **Cost Control:** By delivering incrementally, organizations can stop development when sufficient value has been delivered, avoiding unnecessary spending on low-priority features.

10. How can you transform ER Diagram into relation? Explain with your own suitable example.

[10]

Answer: Transforming ER Diagram into Relational Model:
The process of converting an Entity-Relationship (ER) diagram into a set of relations (tables) is called logical database design. The following rules guide this transformation:

Rule 1: Strong Entity to Relation
Each strong entity type becomes a relation (table). The attributes of the entity become columns of the table. The primary key of the entity becomes the primary key of the table.

Rule 2: Weak Entity to Relation
A weak entity becomes a relation that includes a foreign key referencing the primary key of the identifying strong entity. The primary key of the weak entity is a composite key consisting of the partial key of the weak entity plus the foreign key.

Rule 3: 1:1 Relationship
Add the primary key of one entity as a foreign key in the other entity's relation. It is better to add the foreign key to the entity with total participation.

Rule 4: 1:N Relationship
Add the primary key of the entity on the "1" side as a foreign key in the relation of the entity on the "N" side.

Rule 5: M:N Relationship
Create a new relation (junction/associative entity) that includes the primary keys of both participating entities as foreign keys. These foreign keys together form the primary key of the new relation. Any attributes of the relationship also become columns.

Rule 6: Multivalued Attribute
Create a separate relation for the multivalued attribute. The relation includes the multivalued attribute and a foreign key referencing the primary key of the original entity.

Example — University Database:
Consider an ER Diagram with:
• **Entity: STUDENT** (student_id, name, email, phone)
• **Entity: COURSE** (course_id, title, credits)
• **Entity: DEPARTMENT** (dept_id, dept_name)
• **Relationship: ENROLLS** (M:N between STUDENT and COURSE, attribute: grade)
• **Relationship: BELONGS_TO** (N:1 between STUDENT and DEPARTMENT)

Transformed Relations:
RelationAttributesKeys STUDENTstudent_id, name, email, phone, dept_idPK: student_id, FK: dept_id COURSEcourse_id, title, creditsPK: course_id DEPARTMENTdept_id, dept_namePK: dept_id ENROLLSstudent_id, course_id, gradePK: (student_id, course_id), FK: student_id, course_id

Explanation:
• STUDENT and COURSE are strong entities, so they become separate tables.
• DEPARTMENT is also a strong entity with its own table.
• The BELONGS_TO relationship (N:1) is handled by adding dept_id as a foreign key in the STUDENT table.
• The ENROLLS relationship (M:N) requires a separate ENROLLS table containing both primary keys as

foreign keys, plus the relationship attribute grade.
• The composite primary key of ENROLLS ensures a student can enroll in multiple courses and a course can have multiple students.

11. Why is the project management important? Describe the concept of integrated CASE tools with its applications.

[3+7]

Answer: Importance of Project Management:

1. **Achieving Objectives:** Project management ensures that projects are completed on time, within budget, and according to specifications. Without proper management, projects often fail to meet their goals.
2. **Resource Optimization:** Effective project management allocates resources (human, financial, technical) efficiently, preventing waste and ensuring maximum productivity.
3. **Risk Mitigation:** Project management identifies potential risks early and develops strategies to minimize their impact, reducing the likelihood of project failure.
4. **Quality Control:** Structured project management includes quality checkpoints and standards, ensuring deliverables meet stakeholder expectations.
5. **Stakeholder Satisfaction:** Clear communication and expectation management keep stakeholders informed and satisfied throughout the project lifecycle.
6. **Scope Management:** Prevents scope creep by clearly defining what is included and excluded, and managing change requests formally.
7. **Competitive Advantage:** Organizations with strong project management capabilities can deliver products faster and more reliably than competitors.

Integrated CASE Tools (Computer-Aided Software Engineering):

Integrated CASE tools (I-CASE) are software applications that provide comprehensive support for the entire software development lifecycle (SDLC) within a single, unified environment. Unlike upper CASE (front-end) tools that focus on analysis and design, or lower CASE (back-end) tools that focus on implementation and testing, I-CASE covers all phases from requirements gathering to maintenance.

Concept of Integrated CASE Tools:

I-CASE tools provide a seamless environment where:

- Data entered in one phase is automatically available in subsequent phases
- Changes in one diagram or specification are propagated throughout related artifacts
- A central repository stores all project information (encyclopedia/dictionary)
- Multiple team members can collaborate on the same project simultaneously
- Traceability is maintained from requirements through design to code and testing

Applications of Integrated CASE Tools:

1. **Requirements Management:** Tools like IBM Rational RequisitePro and DOORS help capture, organize, and trace requirements throughout the development process.
2. **Modeling and Design:** Tools like Enterprise Architect, Visual Paradigm, and StarUML support UML modeling (use case, class, sequence, activity diagrams) and automatically generate code skeletons.
3. **Code Generation and Reverse Engineering:** I-CASE tools can generate source code from design models and reverse-engineer code back into design diagrams, maintaining synchronization.
4. **Project Planning and Tracking:** Integrated Gantt charts, resource allocation, and progress tracking help managers monitor project health.
5. **Testing Support:** Test case generation, test execution, defect tracking, and coverage analysis are integrated within the same environment.
6. **Version Control and Configuration Management:** Built-in version control manages changes to all project artifacts, enabling rollback and audit trails.
7. **Documentation:** Automatic generation of technical documentation, user manuals, and reports from the central repository.
8. **Database Design:** Tools like ERwin and PowerDesigner support ER modeling and automatic generation of

database schemas for multiple DBMS platforms.

Examples of Integrated CASE Tools:
- **IBM Rational Suite:** Comprehensive toolset covering requirements, modeling, development, and testing.
- **Microsoft Visual Studio:** IDE with integrated design, coding, debugging, and deployment capabilities.
- **Oracle JDeveloper:** Complete development environment for Java and Oracle applications.
- **Sparx Systems Enterprise Architect:** Full lifecycle modeling tool supporting UML, BPMN, and SysML.

Note: These answers are prepared for educational purposes. Students are encouraged to verify with official textbooks and course materials.